

IN100 Initiation au Python

Contrôle continu 2

Étudiant :	Heure Début	Note (/5)
Correcteur :	Heure Fin	

Consignes: Vous avez 20 minutes pour coder la solution. Seules les types et les instructions vues en cours sont autorisées. Tout usage d'une fonction venant d'une bibliothèque qui n'a pas été vue en cours ne sera pris en compte.

Sujet: Durant un cours de sport, un professeur souhaite organiser un tournoi de tennis de table. Sa classe comporte 26 élèves, qu'il numérote de 1 à 26. Il dispose de 13 tables sur lesquels les élèves s'affrontent. On représente les terrains par une liste de tables, et chaque table par un tuple constituant une paire d'élèves. Par exemple, une disposition de départ des élèves est la suivante:

$[(1,2), (3,4), (5,6), \dots, (23,24), (25,26)]$

1. Une fois un match terminé sur une table, on souhaite que le premier élèves du tuples correspondant soit le vainqueur. Écrire une fonction Python `match` qui prend en entrée un tuple d'élèves, et choisit un gagnant au hasard et renvoie un tuple adapté au résultat du match.
2. Lorsqu'un élève gagne, il monte à la table voisine d'indice supérieur. Lorsqu'un élève perd, il descend à la table voisine d'indice inférieur. Le perdant de la table d'indice 0 ne change pas de table, et il en va de même pour le gagnant de la table d'indice 12. Écrire une fonction Python `round` qui prend en entrée une liste de tuples représentant les tables, et qui simule un match sur chaque table et renvoie la liste des tables après que les élèves se soient déplacés.

Pour générer aléatoirement des nombres entiers, nous rappelons qu'on peut utiliser la fonction `random.randint(a,b)` qui renvoie un nombre entier entre a et b .

Exemple d'usage:

```
import random
```

```
print(random.randint(2,15))
```

Remarques étudiant :

Remarques correcteur :

IN100 Initiation au Python

Contrôle continu 2

Étudiant :	Heure Début	Note (/5)
Correcteur :	Heure Fin	

Consignes: Vous avez 20 minutes pour coder la solution. Seules les types et les instructions vues en cours sont autorisées. Tout usage d'une fonction venant d'une bibliothèque qui n'a pas été vue en cours ne sera pris en compte.

Sujet: On considère la suite (u_n) définie par :

$$\begin{cases} u_0 = 2 \\ u_{n+1} = 3u_n + 1 \end{cases} \text{ pour tout } n \geq 0$$

Les termes de cette suite peuvent être calculés progressivement à partir de la valeur initiale u_0 .

Exemple :

$$u_1 = 3 \times 2 + 1 = 7 \quad u_2 = 3 \times 7 + 1 = 22$$

1. Écrire une fonction `terme_n_suite` qui prend en argument :

- un entier initial u_0 ,
- un entier n .

Cette fonction doit retourner la valeur du terme u_n de la suite.

2. Écrire une fonction qui détermine le premier rang n pour lequel le terme u_n est strictement supérieur à un entier donné M .

3. Tester les fonctions précédentes avec plusieurs valeurs de n et de M .

Remarques étudiant :
Remarques correcteur :

IN100 Initiation au Python

Contrôle continu 2

Étudiant :	Heure Début	Note (/5)
Correcteur :	Heure Fin	

Consignes: Vous avez 20 minutes pour coder la solution. Seules les types et les instructions vues en cours sont autorisées. Tout usage d'une fonction venant d'une bibliothèque qui n'a pas été vue en cours ne sera pris en compte.

Sujet: On définit une suite $(S_n)_{n \in \mathbb{N}}$ de chaînes de caractères par :

- $s_0 = \text{"a"}$,
- pour tout $n \geq 0$,

$$s_{n+1} = s_n + \text{"b"} + s_n,$$

où le symbole $+$ désigne la concaténation de chaînes de caractères.

Questions:

1. Écrire une fonction Python `calcul_s(n)` qui calcule et renvoie la chaîne de caractères s_n .
2. On rappelle qu'une chaîne de caractères est un *palindrome* si elle est égale à son renversé. Écrire une fonction Python qui prend en entrée un entier `n` et permet de tester si la chaîne s_n pour est un palindrome.

Remarques étudiant :
Remarques correcteur :

IN100 Initiation au Python

Contrôle continu 2

Étudiant :	Heure Début	Note (/5)
Correcteur :	Heure Fin	

Consignes: Vous avez 20 minutes pour coder la solution. Seules les types et les instructions vues en cours sont autorisées. Tout usage d'une fonction venant d'une bibliothèque qui n'a pas été vue en cours ne sera pris en compte.

Sujet: On souhaite simuler une file d'attente (par exemple à un guichet). La file d'attente sera représentée par une liste Python contenant les noms des clients.

1. Écrire une fonction **ajouter_client(file_attente, nom)** qui prend en paramètre une liste **file_attente** et une chaîne de caractères **nom** et ajoute le client **nom** à la fin de la file.
2. Écrire une fonction **servir_client(file_attente)** qui prend en paramètre une liste **file_attente**. Si la file est vide, on affiche le message "**Aucun client à servir**" et on renvoie **None**. Sinon, on retire le premier client de la file et on renvoie son nom.
3. Écrire un programme principal qui :
 - initialise une file d'attente vide,
 - demande à l'utilisateur combien de clients arrivent successivement,
 - pour chaque client, demande son nom et appelle **ajouter_client**,
 - appelle ensuite **servir_client** une première fois et affiche le client servi,
 - affiche enfin le contenu restant de la file d'attente.

Remarques étudiant :
Remarques correcteur :

IN100 Initiation au Python

Contrôle continu 2

Étudiant :	Heure Début	Note (/5)
Correcteur :	Heure Fin	

Consignes: Vous avez 20 minutes pour coder la solution. Seules les types et les instructions vues en cours sont autorisées. Tout usage d'une fonction venant d'une bibliothèque qui n'a pas été vue en cours ne sera pris en compte.

Sujet: On construit un collier de perle avec des perles bleu et rouge. On modélise un collier par une liste de perles, et une perle par sa couleur 'B' ou 'R'. Cependant chaque collier forme une boucle, c'est-à-dire que la première et la dernière perle sont considérées comme voisines. Du coup, un collier peut être représenté par plusieurs listes.

Exemple: La liste ['B','R','R','B','R'] représente un collier à 5 perles, qui peut aussi être représenté par ['R','R','B','R','B']).

1. Écrire une fonction Python `verifier_identiques` qui prend en entrée deux listes de perles et qui renvoie `True` si elles correspondent au même collier.
2. On dira qu'un collier est alterné si deux perles voisines n'ont jamais la même couleur. Écrire une fonction Python `rendre_alterné_plus` qui modifie une liste de perles pour la rendre alternée, en lui ajoutant des perles. Si nécessaire, on pourra "ajouter des perles" entre deux perles consécutifs du collier.

Remarques étudiant :

Remarques correcteur :